



中国科学技术大学
University of Science and Technology of China

语言的描述：词法和语法

《编译原理和技术(H)》

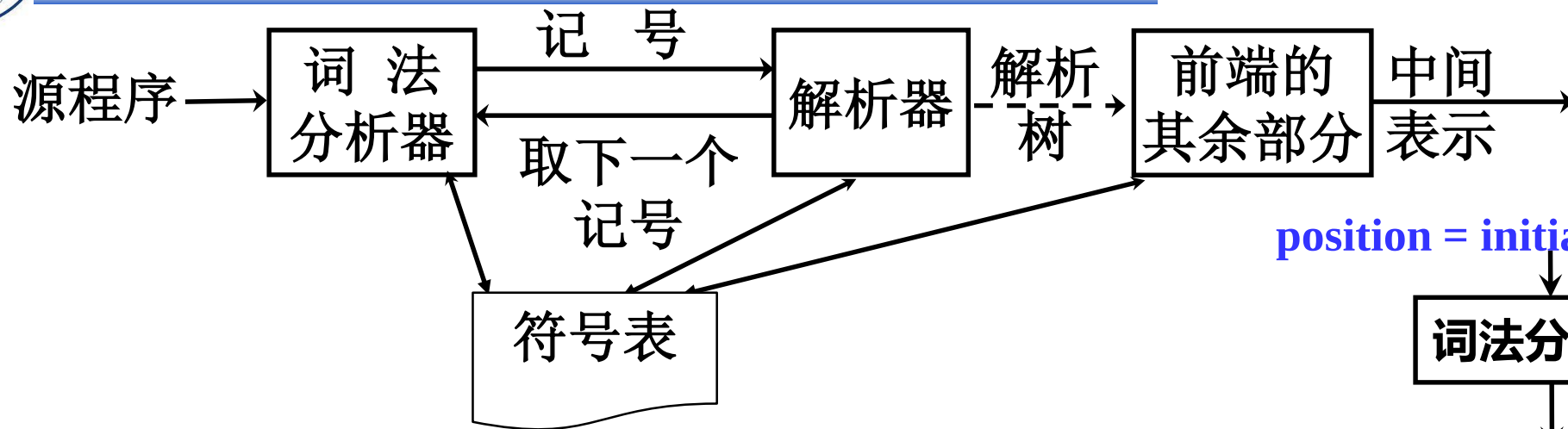
张昱

yuzhang@ustc.edu.cn

中国科学技术大学
计算机科学与技术学院



内容【教材第2、3章】

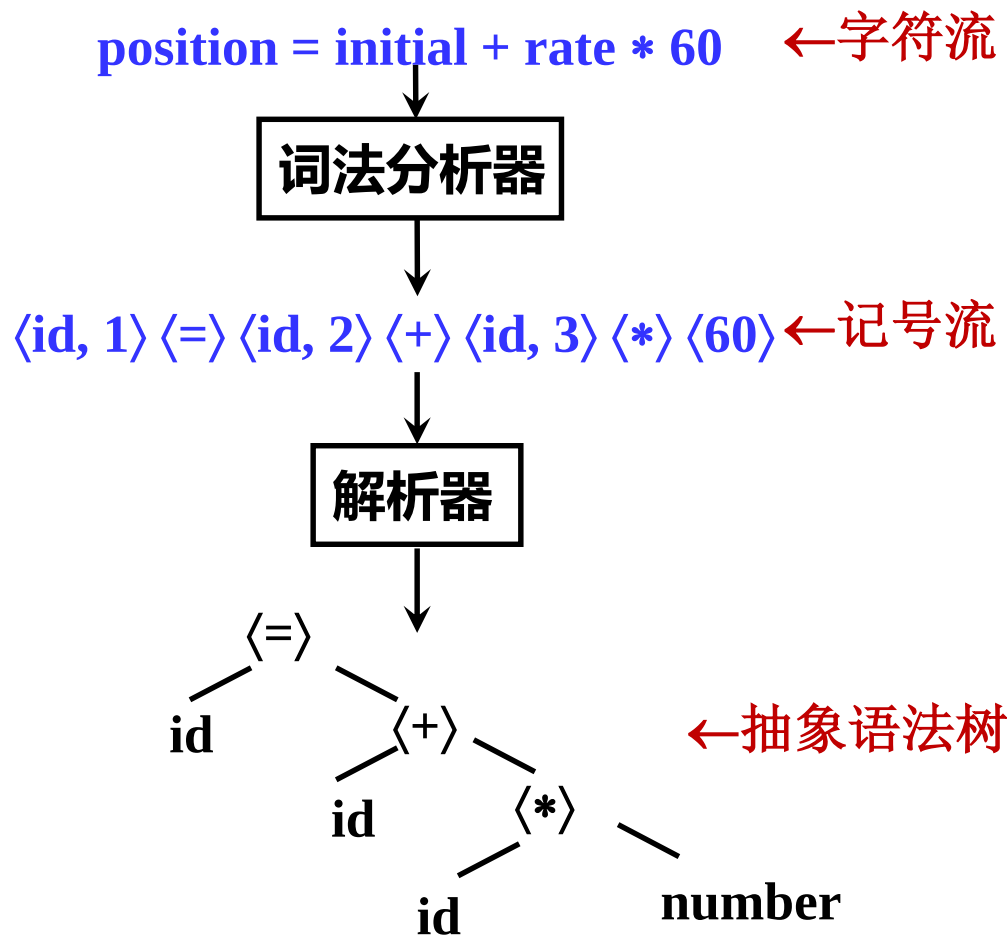


□ 词法描述与识别：字符流→记号(词元)流

- 词法的描述：正规式；
- 记号的识别：转换图（有限自动机：NFA、DFA）

□ 语法描述与分析：记号流→语法树

- 语法的描述：上下文无关文法；
- 解析技术：自上而下推导、自下而上归约





中国科学技术大学
University of Science and Technology of China

语言的描述：词法和语法

《编译原理和技术(H)》



词法记号及属性

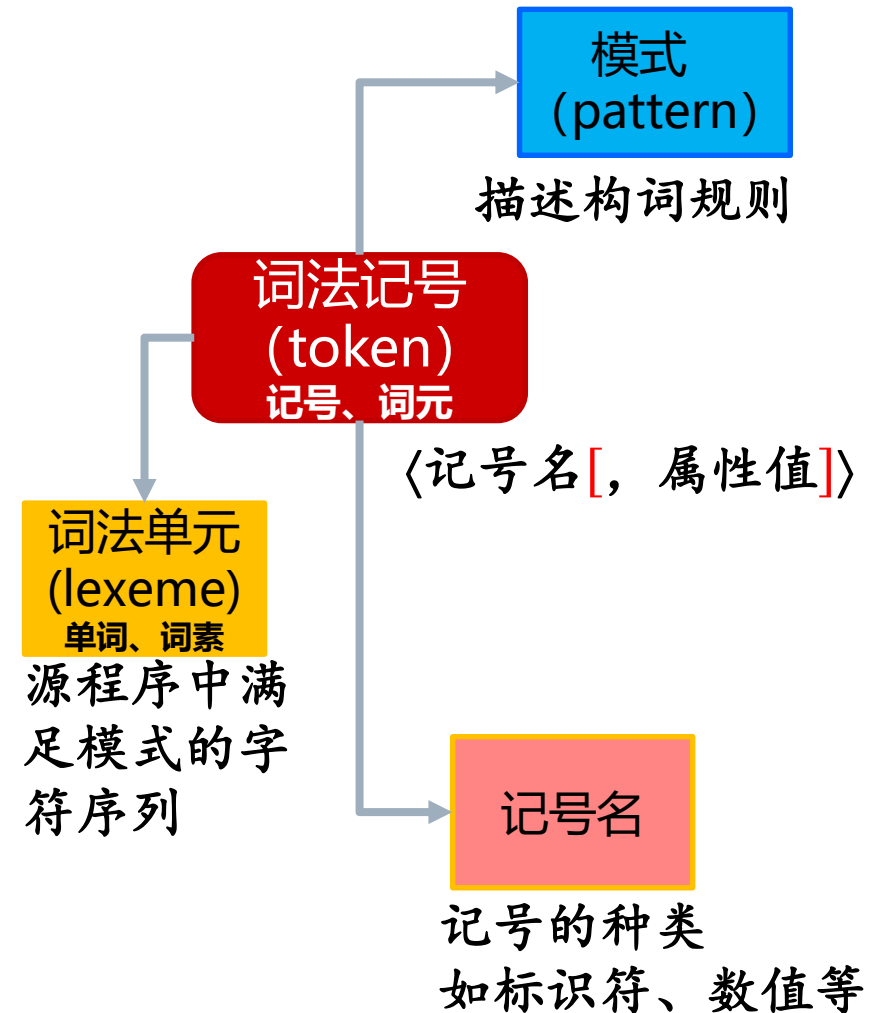
- ☐ 词法单元(lexeme, 词素)
- ☐ 记号(token)
- ☐ 模式(pattern)



词法记号、词法单元、模式

记号名	词法单元实例	模式的非形式描述
if	if	字符i, f
for	for	字符f, o, r
relop	<, <=, =, ...	< 或 <= 或 = 或 ...
id	sum, count, D5	由字母开头的字母数字串
number	3.1, 10, 2.8 E12	任何数值常数
literal	“seg. error”	引号“和”之间任意不含引号本身的字符串
ws	换行符	换行符\n

空白字：如空格、\t、换行
无意义，被丢弃，不提供给语法分析器





词法记号的属性

position = initial + rate * 60 的记号

⟨记号名[, 属性值]⟩:

⟨**id**, 指向符号表中position条目的指针⟩

⟨**assign_op**⟩

⟨**id**, 指向符号表中initial条目的指针⟩

⟨**add_op**⟩

⟨**id**, 指向符号表中rate条目的指针⟩

⟨**mul_op**⟩

⟨**number**, 整数值60⟩

符 号 表

1	position	...
2	initial	...
3	rate	...

Lexeme
词法单元/单词/词素



词法定义中的问题

□ 关键字 ≠ 保留字

- **关键字(keyword)**: 有专门的意义和用途, 如if、else
- **保留字**: 有专门的意义, 不能当作一般的标识符使用
例如, C语言中的关键字是保留字

□ 历史上词法定义中的一些问题

- 忽略空格带来的困难, 例如 Fortran

DO 8 I = 3.75 等同于 DO8I = 3.75

DO 8 I = 3,75

空格不是
分隔符

- 关键字不保留

IF THEN THEN THEN=ELSE; ELSE ...



中国科学技术大学
University of Science and Technology of China

词法记号的描述与识别

- 描述：正规式
- 识别：转换图



□ 术语

■ **字母表**：符号的有限集合，例： $\Sigma = \{0, 1\}$ 、ASCII、Unicode

■ **串**：符号的有穷序列，例：0110, ε

■ **语言**：字母表 Σ 上的一个串集

$\{\varepsilon, 0, 00, 000, \dots\}$, $\{\varepsilon\}$, $\emptyset$

■ **句子**：属于语言的串

□ 串的运算

■ **连接（积）** xy , $s\varepsilon = \varepsilon s = s$

■ **幂** s^0 为 ε , s^i 为 $s^{i-1}s$ ($i > 0$)

词法单元是源程序中的字符序列（字符串）
字符串集合由称为模式的规则来描述

串s的长度是出现在s中符号的个数

注意区别：

ε , $\{\varepsilon\}$, $\emptyset$

空串

非空
集合

空集

优先级：

幂 > 连接



□ 语言的运算

语言表示字母表上的一个串集

- 并: $L \cup M = \{s \mid s \in L \text{ 或 } s \in M\}$
- 连接: $LM = \{st \mid s \in L \text{ 且 } t \in M\}$
- 幂: L^0 是 $\{\epsilon\}$, L^i 是 $L^{i-1}L$
- 闭包: $L^* = L^0 \cup L^1 \cup L^2 \cup \dots$
- 正闭包: $L^+ = L^1 \cup L^2 \cup \dots$

优先级:
幂 > 连接 > 并
均是自左向右结合

□ 例

$L: \{A, B, \dots, Z, a, b, \dots, z\}, D: \{0, 1, \dots, 9\}$

$L \cup D, LD, L^6, L^*, L(L \cup D)^*, D^+$



正规式(regular expression)

正规式(正则表达式)用来表示简单的语言, 叫做正规集

正规式	定义的语言	备注
ε	$\{\varepsilon\}$	
a	$\{a\}$	$a \in \Sigma$
(r)	$L(r)$	r 是正规式
$(r) \mid (s)$	$L(r) \cup L(s)$	r 和 s 是正规式
$(r)(s)$	$L(r)L(s)$	r 和 s 是正规式
$(r)^*$	$(L(r))^*$	r 是正规式

$((a)(b)^*) \mid (c)$ 可以写成 $ab^* \mid c$

优先级:
闭包 * > 连接 > 选择 $\mid$



正规式遵守的一些代数定律

定律	描述
$r s = s r$	是可交换的
$r (s t) = (r s) t$	是可结合的
$(rs)t = r(st)$	连接是可结合的
$r(s t) = rs rt; (s t)r = sr tr$	连接对 是可分配的
$\varepsilon r = r\varepsilon = r$	ε 是连接的恒等元素
$r^* = (r \varepsilon)^*$	ε 肯定出现在一个闭包中
$r^{**} = r^*$	*是幂等的



正规式举例

□ $\Sigma = \{a, b\}$

- $a \mid b$ $\{a, b\}$
- $(a \mid b)(a \mid b)$ $\{aa, ab, ba, bb\}$
- $aa \mid ab \mid ba \mid bb$ $\{aa, ab, ba, bb\}$
- a^* 由字母 a 构成的所有串的集合
- $(a \mid b)^*$ 由 a 和 b 构成的所有串的集合

□ 复杂的例子

$(00 \mid 11 \mid ((01 \mid 10)(00 \mid 11)^*(01 \mid 10)))^*$

句子: 01001101000010000010111001



正规定义(regular definition)

- 对正规式命名, 使正规式表示简洁

$$d_1 \rightarrow r_1$$

$$d_2 \rightarrow r_2$$

...

$$d_n \rightarrow r_n$$

自底向上定义
Bottom-up

- 各个 d_i 的名字都不同, 是新符号, 不在字母表 Σ 中
- 每个 r_i 都是 $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$ 上的正规式
- C语言的标识符是字母、数字和下划线组成的串

$$\text{letter_} \rightarrow A | B | \dots | Z | a | b | \dots | z | _$$

$$\text{digit} \rightarrow 0 | 1 | \dots | 9$$

$$\text{id} \rightarrow \text{letter_}(\text{letter_} | \text{digit})^*$$



正规定义举例

□ 无符号数集合，例1946, 11.28, 63E8, 1.99E-6

digit $\rightarrow 0 \mid 1 \mid \dots \mid 9$

digits $\rightarrow \text{digit digit}^*$

optional_fraction $\rightarrow \text{.digits} \mid \varepsilon$

optional_exponent $\rightarrow (E (+ \mid - \mid \varepsilon) \text{digits}) \mid \varepsilon$

number $\rightarrow \text{digits optional_fraction optional_exponent}$



正规定义举例

□ 无符号数集合，例1946, 11.28, 63E8, 1.99E-6

$\text{digit} \rightarrow 0 \mid 1 \mid \dots \mid 9$

简记为 $[0-9]$ --- 字符组

$\text{digits} \rightarrow \text{digit digit}^*$

$\text{optional_fraction} \rightarrow \text{.digits} \mid \varepsilon$

$\text{optional_exponent} \rightarrow (E (+ \mid - \mid \varepsilon) \text{digits}) \mid \varepsilon$

$\text{number} \rightarrow \text{digits optional_fraction optional_exponent}$

□ 简化的表示

$\text{number} \rightarrow \text{digit}^+ (\text{.digit}^+)? (E(+|-)? \text{digit}^+)?$

注意区分? 和 *

? 表示0个或1个, *表示0个或多个, +表示1个或多个



中国科学技术大学
University of Science and Technology of China

词法记号的描述与识别

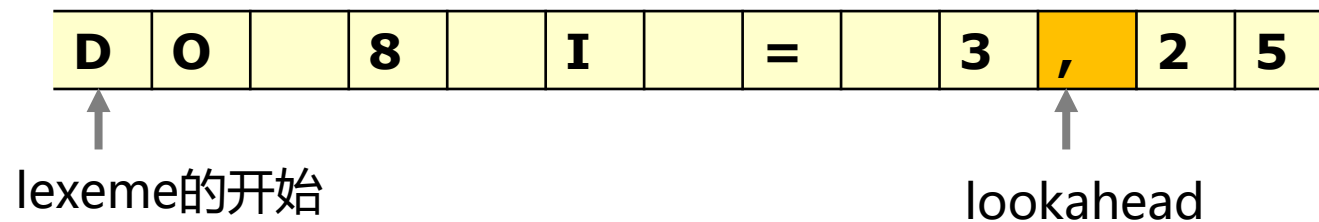
- 描述：正规式
- 识别：转换图



词法分析

□ 词法分析

- 从左到右读取输入串，每次识别出一个token实例
- 可能需要“lookahead”来判断当前是否是token的结尾或下一个token的开始（尤其是对Fortran语言）



DO 8 I = 3

无法判断，需
lookahead

DO 8 I = 3.25

点→DO8I为id

DO 8 I = 3, 25

逗号→DO为关键字



词法分析

□ 词法分析

- 从左到右读取输入串，每次识别出一个token实例
- 可能需要“lookahead”来判断当前是否是一个token的结尾、下一个token的开始（尤其是在Fortran语言中）
- 可能需要结合上下文来识别是否是关键字（当关键字不是保留字时）

```
if (then > else) then  
    then = else  
else  
    else = then  
endif
```

需要结合上下文
识别是否是关键字



词法分析器：实现

一个词法分析器的实现必须做两件事

1. 识别子串并对应到 tokens
2. 返回token的值或词法单元(lexeme, 词素)
 - 词法单元是子串 (token的实例)

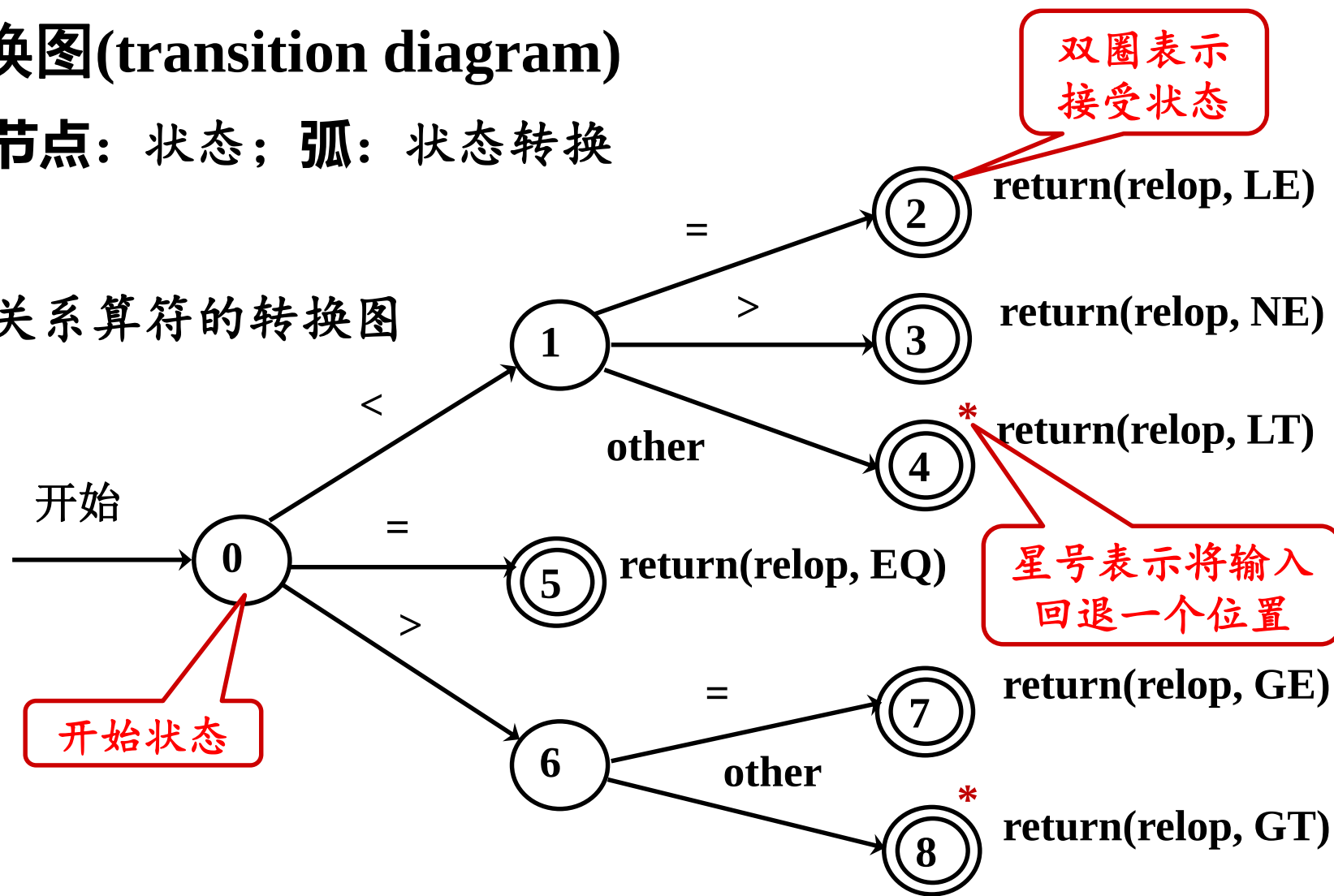


词法记号的识别：状态转换图

□ 转换图(transition diagram)

■ 节点：状态；弧：状态转换

■ 关系算符的转换图





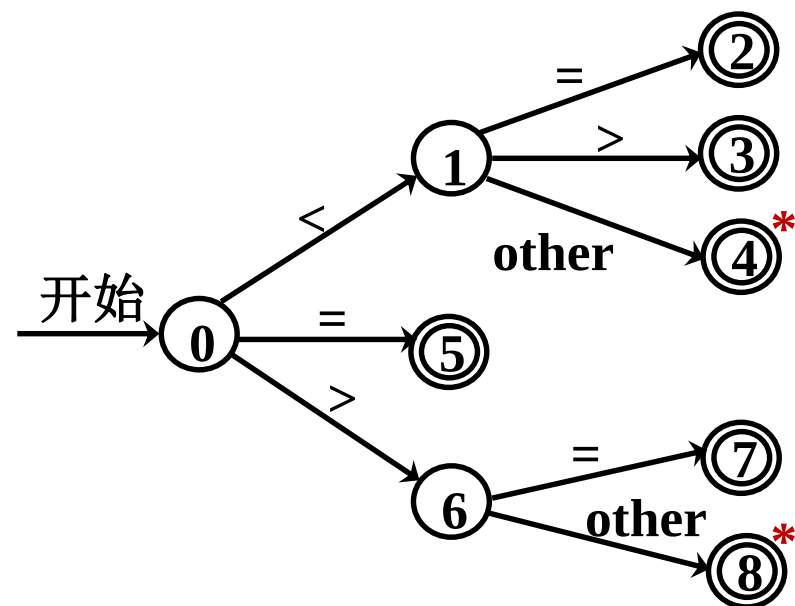
基于转换图的词法分析

例：relop的转换图的概要实现

```
TOKEN getRelop() {  
    TOKEN retToken = new(RELOP);  
    while (1) {  
        switch (state) {  
            case 0: c = nextChar();  
                if (c == '<') state = 1;  
                else if (c == '=') state = 5;  
                else if (c == '>') state = 6;  
                else fail();  
                break;  
            case 1: ...  
            ...  
            case 8: retract();  
                retToken.attribute = GT;  
                return(retToken);  
        }  
    }  
}
```

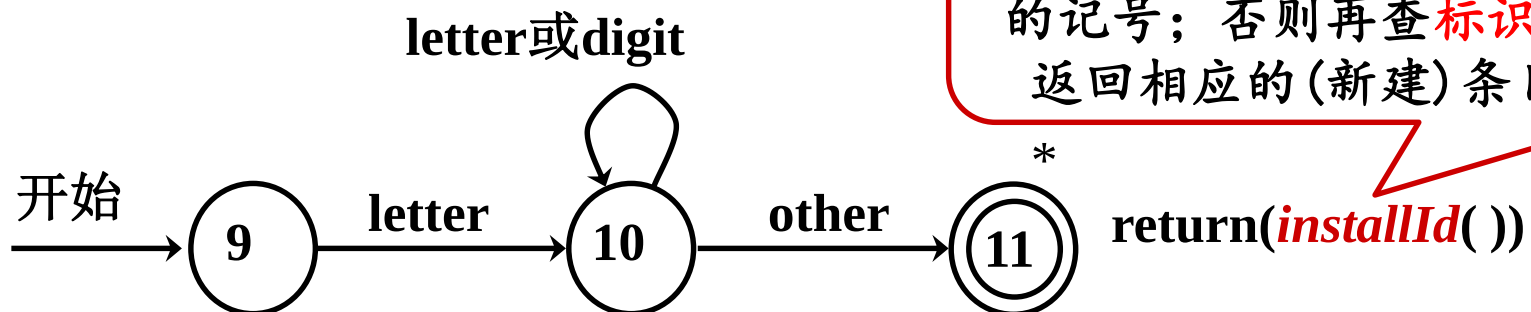
出错处理，要
能从错误恢复

回退



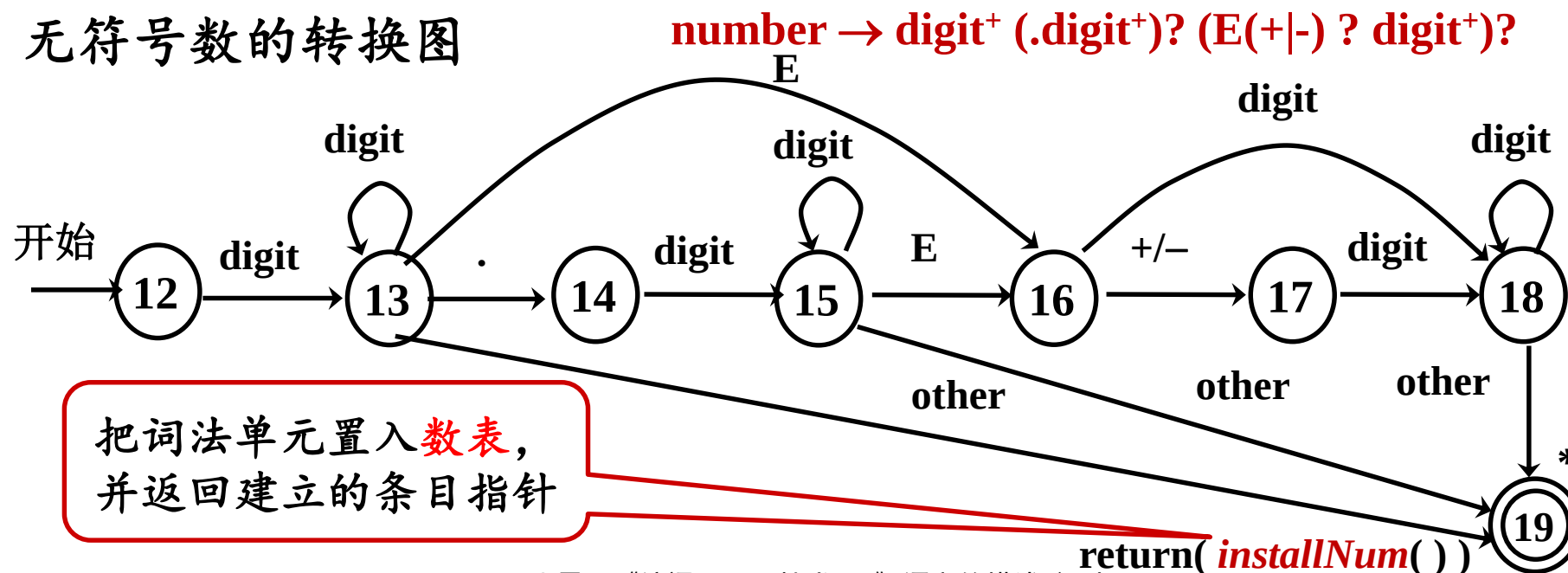
转换图

标识符和关键字的转换图



先查看**关键字表**，若当前词法单元构成关键字，则返回相应的记号；否则再查**标识符表**，返回相应的(新建)条目指针

无符号数的转换图



把词法单元置入**数表**，并返回建立的条目指针

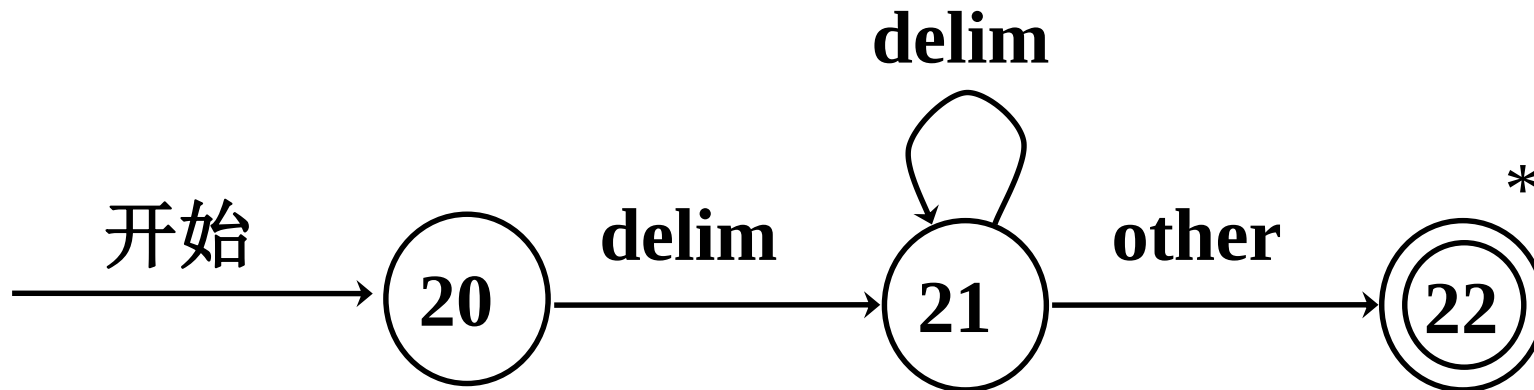


转换图

□ 空白的转换图

delim → **blank** | **tab** | **newline**

ws → **delim**⁺





词法分析中的冲突及解决

$R = \text{Whitespace} \mid \text{Integer} \mid \text{Identifier} \mid '+'$

分析 “foo+3”

→ “f” 匹配 R , 更精确地说是 **Identifier**

→ 但是 “fo” 也匹配 R , “foo” 也匹配, 但 “foo+” 不匹配

如何处理输入? 如果

$x_1 \dots x_i \in L(R)$ 并且 $x_1 \dots x_k \in L(R)$, 假设 $i < k$

则可按“**Maximal munch**”规则来选择:

→ 选择匹配 R 的最长前缀, 即选择 $x_1 \dots x_k$

最长匹配规则在实现时: lookahead, 不符合则回退



词法分析：分类的不确定性

$R = \text{Whitespace} \mid \text{'new'} \mid \text{Integer} \mid \text{Identifier}$

分析 “new foo”

- “new” 匹配 R , 更精确地说是 ‘new’
- 但是也匹配 Identifier, 此时该选哪个?

一般地, 如果 $x_1 \dots x_i \in L(R_j)$ 和 $x_1 \dots x_i \in L(R_k)$

规则: 选择先列出的模式 (R_j 如果 $j < k$)

- 必须将 ‘new’ 列在 Identifier 的前面



词法错误

词法分析器对源程序采取非常局部的观点

- 例：难以发现下面的错误

`fi (a == f (x) ) ...`

- 在实数是“**数字串.数字串**”格式下，可以发现下面的错误

123.x

注：数字串长度不小于1

- 紧急方式的错误恢复

删掉当前若干个字符，直至能读出正确的记号

- 错误修补

进行增、删、替换和交换字符的尝试



例题 1

写出语言“所有相邻数字都不相同的非空数字串”的正规定义。

123031357106798035790123

解答：

$answer \rightarrow (0 \mid no_0 \ 0) (no_0 \ 0)^* (no_0 \mid \varepsilon) \mid no_0$

$no_0 \rightarrow (1 \mid no_0-1 \ 1) (no_0-1 \ 1)^* (no_0-1 \mid \varepsilon) \mid no_0-1$

...

$no_0-8 \rightarrow 9$

将这些正规定义逆序排列就是答案



中国科学技术大学
University of Science and Technology of China

有限自动机

- 描述分析器：NFA、DFA
(对转换图的形式定义和分类)



有限自动机

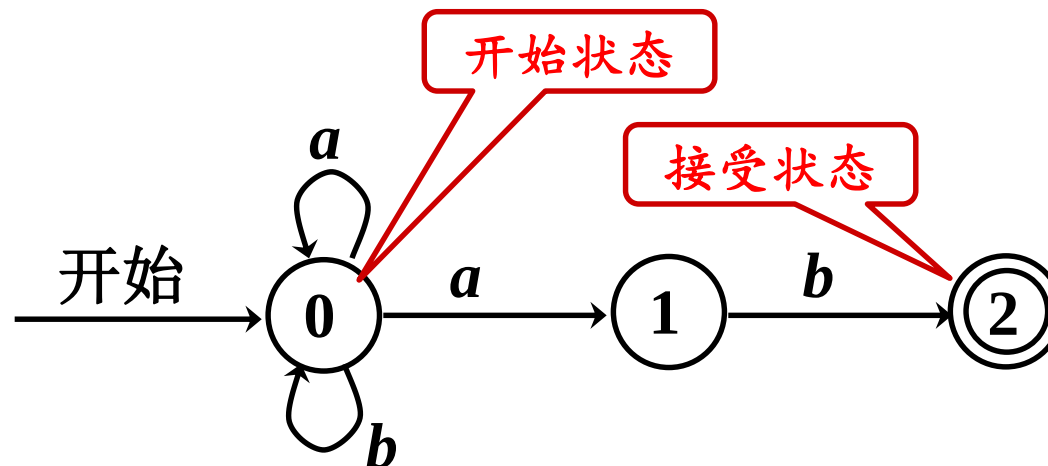
□ 不确定的有限自动机 (NFA)

一个数学模型，它包括：

(nondeterministic finite automaton)

- 1、有限的状态集合 S
- 2、输入符号集合 Σ
- 3、转换函数 $move : S \times (\Sigma \cup \{\epsilon\}) \rightarrow P(S)$
- 4、状态 s_0 是唯一的开始状态
- 5、 $F \subseteq S$ 是接受状态集合

识别语言
 $(a|b)^*ab$
的NFA



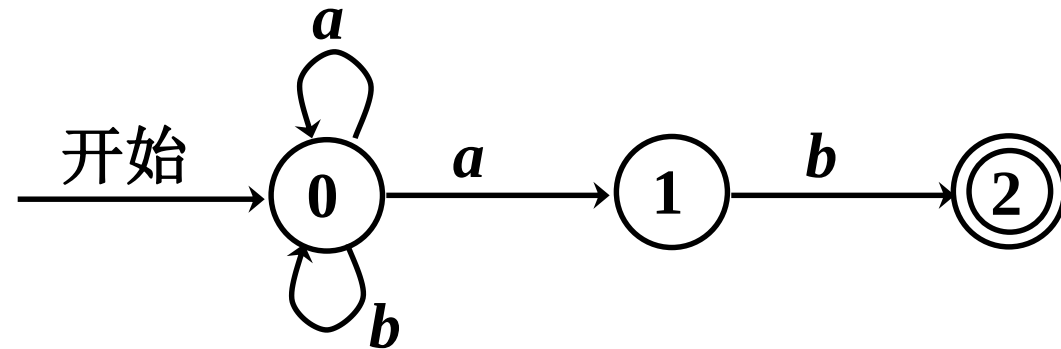


转换表

□ NFA的转换表

	输入符号	
	a	b
0	$\{0, 1\}$	$\{0\}$
1	$\emptyset$	$\{2\}$
2	$\emptyset$	$\emptyset$

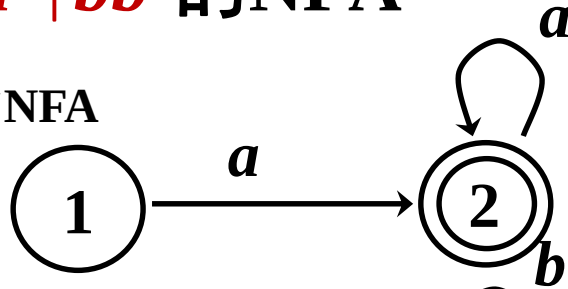
识别语言
 $(a|b)^*ab$
的NFA



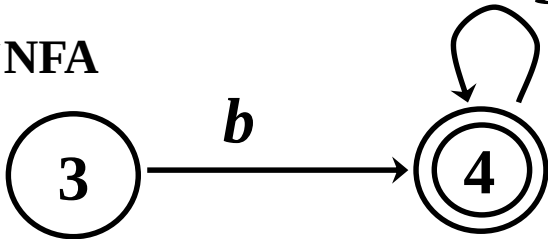
例题3

识别 $aa^* | bb^*$ 的NFA

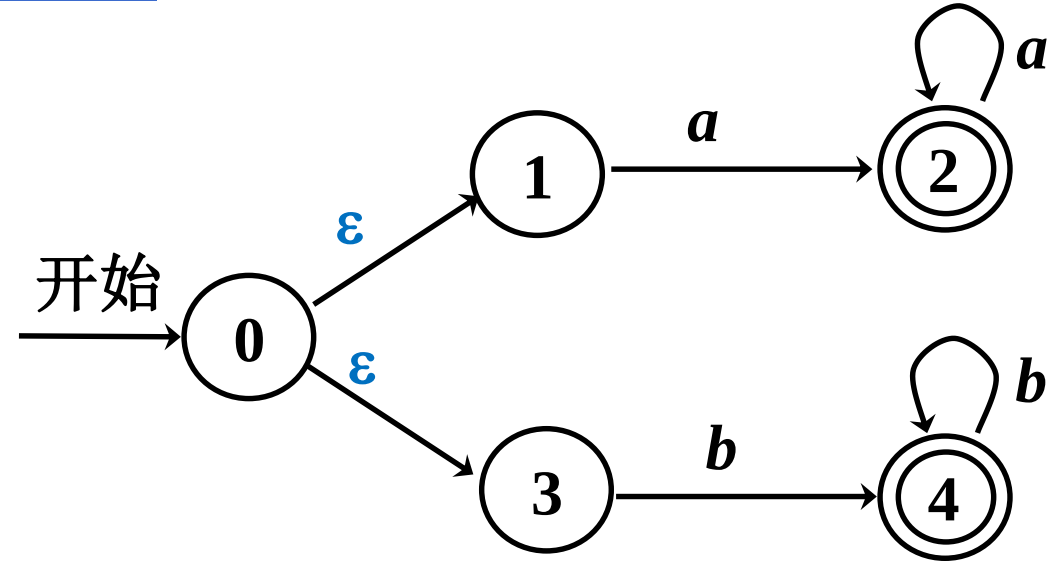
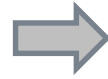
识别 aa^* 的NFA



识别 bb^* 的NFA

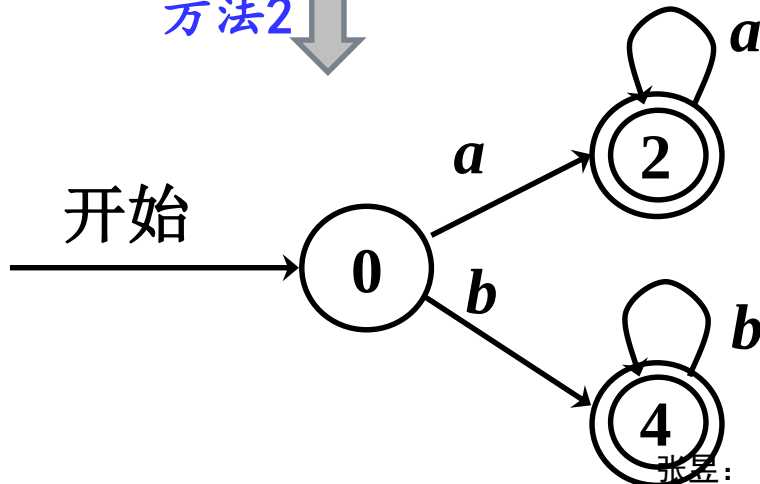


方法1



输入串 aaa 由开始状态0、到1、2、2、2的路径来接受
或者由开始状态0、到3、回退到0，再到1、2、2、2的路径来接受

方法2



合并两个子NFA的开始状态1和3，设为0



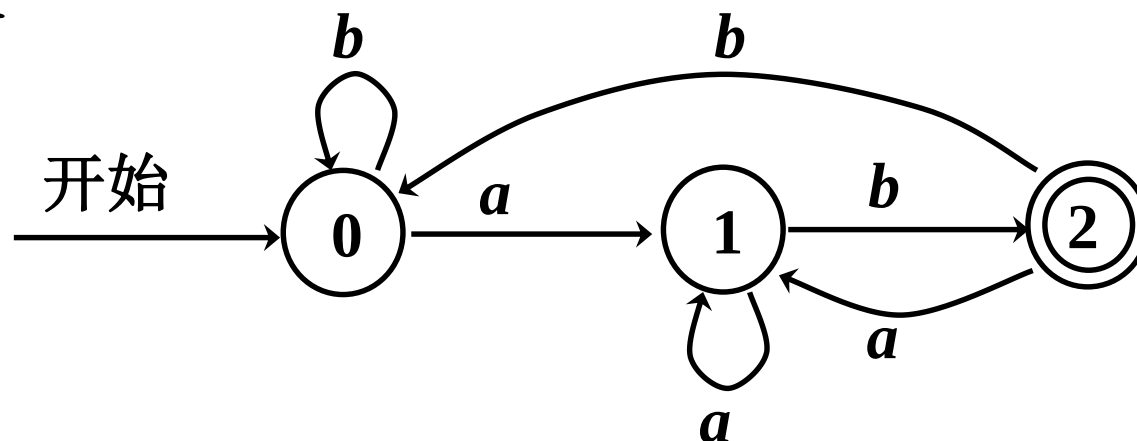
确定的有限自动机

□ 确定的有限自动机 (DFA)

一个数学模型，它包括：

- 1、有限的状态集合 S
- 2、输入符号集合 Σ
- 3、转换函数 $move : S \times \Sigma \rightarrow S$ ，且可以是部分函数
- 4、状态 s_0 是唯一的开始状态
- 5、 $F \subseteq S$ 是接受状态集合

识别语言
 $(a|b)^*ab$
的DFA





NFA vs. DFA

□ 主要差异在转换函数

■ NFA: $move : S \times (\Sigma \cup \{\epsilon\}) \rightarrow P(S)$

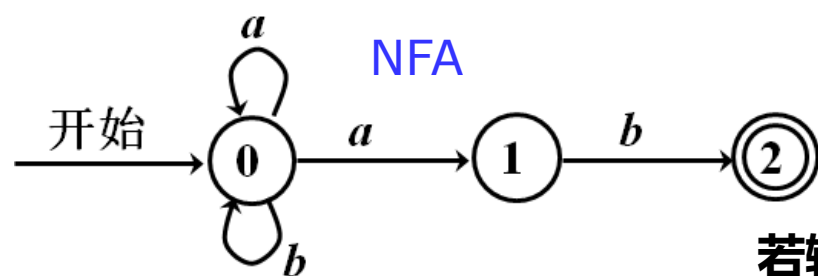
效率低

□ 要识别一个token, 需要对多种可能的路径**试探+失败回退**

■ DFA: $move : S \times \Sigma \rightarrow S$, 可以是**部分函数**

快速高效

□ 对于面临的 Σ 中的符号, 状态转换是明确的



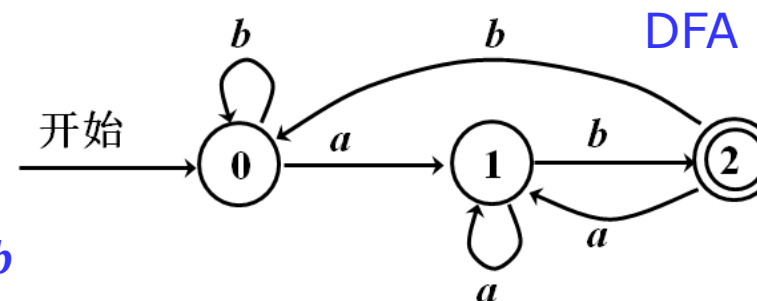
识别语言
 $(a|b)^*ab$

若输入串是 ab

0-(a)→0-(b)→0-结束符
0←回退b

0←回退a

0-(a)→1-(b)→2-结束(接受状态)



0-(a)→1-(b)→2-结束(接受状态)

但是, 由正规式不易构造DFA



例题2

构造一个DFA，它能识别{0,1}上能被5整除的二进制数。

解答

	已读过	尚未读	已读部分的值
某时刻	101	0111000	5
读进0	1010	111000	$5 \times 2 = 10$
读进1	10101	11000	$10 \times 2 + 1 = 21$

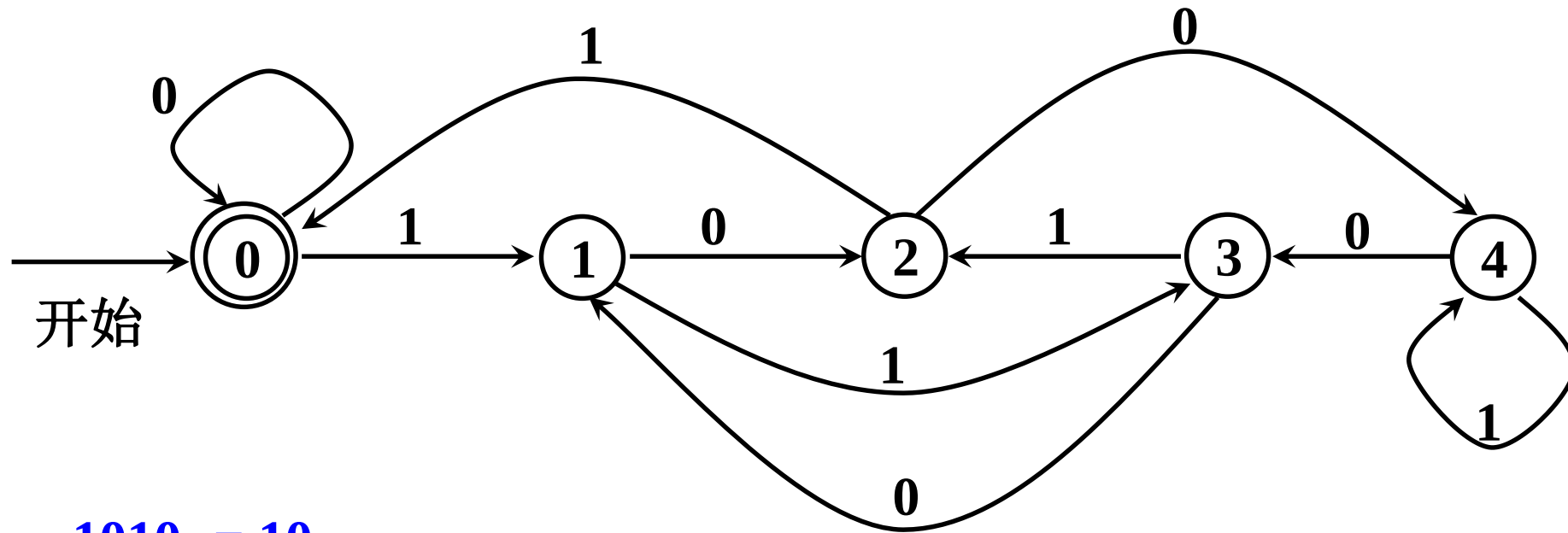
引入5个状态即可，分别代表已读部分的值除以5的余数



例题2

构造一个DFA，它能识别 $\{0,1\}$ 上能被5整除的二进制数。

解答



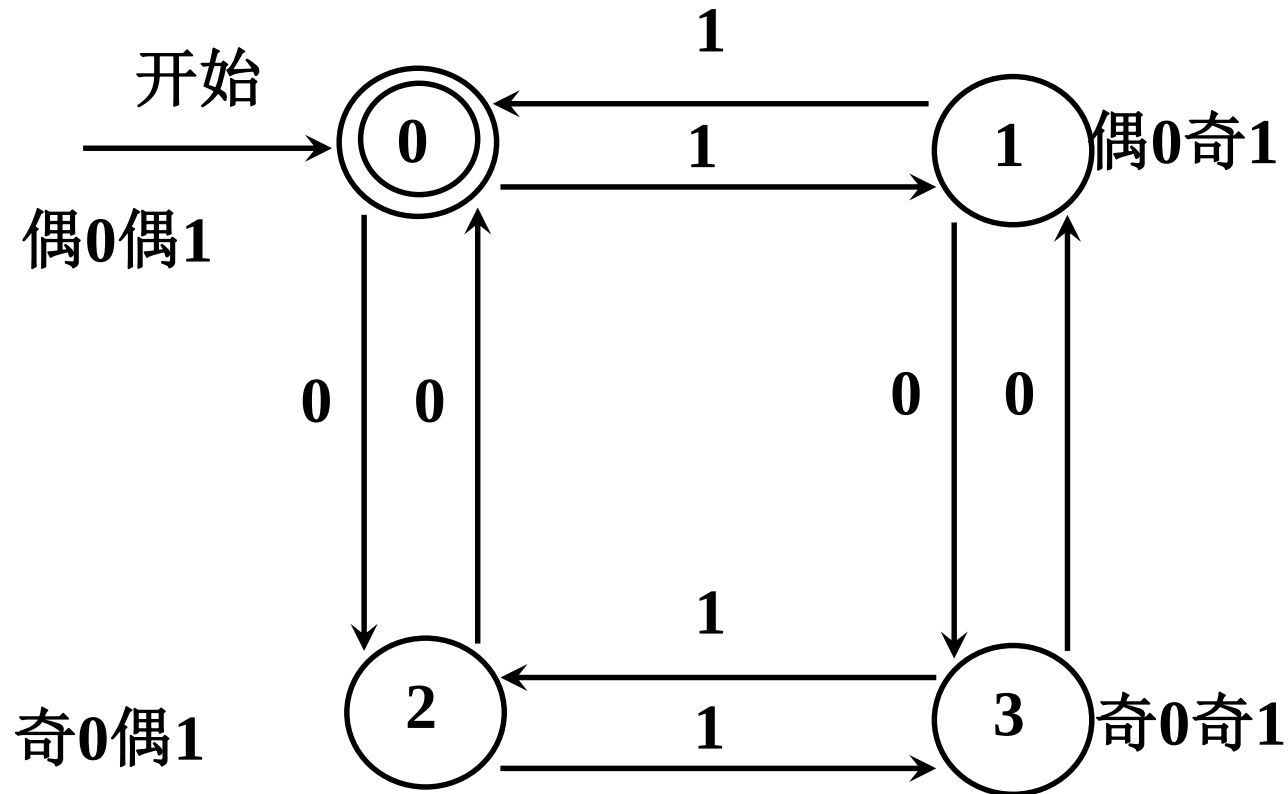
$$1010_2 = 10_{10}$$

$$111_2 = 7_{10}$$



例题3

构造一个DFA，它能接受0和1的个数都是偶数的字符串。





中国科学技术大学
University of Science and Technology of China

大模型的分词技术



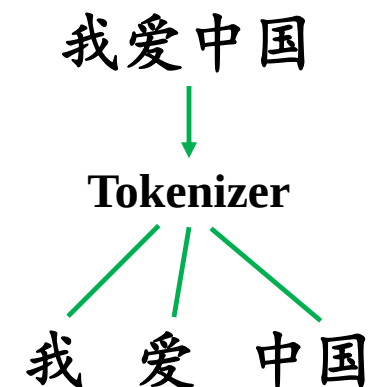
从词法分析到 LLM Tokenizer

□ 在大语言模型（LLM）中，Tokenizer 扮演了类似词法分析器的角色

- 输入：自然语言（甚至多模态输入，如文本、图片描述、emoji符号）
- 输出：tokens，即模型的最小处理单元

□ LLM的分词特点

- **统计驱动**：**分词规则**不是人工定义的，而是从大规模语料中训练**学习**得到
- **子词分词**（subword tokenization）：将频繁出现的字符串作为一个token，将**不常见的词拆分**成更小的、有意义的子词甚至字符。
- 文本Tokenizer的主流算法
 - [BPE](#)(OpenAI GPT系列使用)
 - [WordPiece](#)(Google BERT使用)
 - [SentencePiece](#)(Google的T5、DeBERTa使用)





分词算法：BPE (Byte Pair Encoding)

□ 起源

最初是一种数据压缩算法(用于减少文件大小)，后来被NLP借用来做子词分词

□ 思想

- 从最小单元开始（通常是字符）
- 反复合并高频的相邻单元对；逐步得到越来越大的“子词单元”，作为Tokenizer 的词汇表

BPE的过程就像在学习一个新词：

一开始只认识“e”和“s”，当“es”出现次数足够多时，就把它当成整体来记；然后“es”和“t”又经常出现，被合并成“est”，最终可能把“newest”当作一个完整的词法单元



分词算法：字节级 BPE (Byte-level BPE)

□ BPE的不足

- BPE分词以“字符”为基本单位，对于纯英文文本效果很好
- 但对于中文、日文、甚至emoji等非拉丁字符，将Unicode所有字符都纳入词汇表，**词汇表会变得非常庞大**，且仍然无法覆盖所有可能的字符。

词汇表（Vocabulary）是一个从**Token字符串到ID的映射字典**。其大小是关键超参数（通常在几万到十几万之间）。

□ 字节级BPE

任何文本都是由字节构成，故可以“字节”为基本单位进行BPE分词。
从而，无论是中文、日文还是emoji，都视为字节序列。



示例：Qwen3系列模型的Tokenizer

- 英文句子：Hello, I love University of Science and Technology of China
- 分词结果：['Hello', ',', 'ĠI', 'Ġlove', 'ĠUniversity', 'Ġof', 'ĠScience', 'Ġand', 'ĠTechnology', 'Ġof', 'ĠChina']

原句中的单词和标点符号合理地被拆分成了多个token。

其中的字符 `Ġ` 表示空格，是BPE分词实现中的特点。

- 中文句子：我爱中国科学技术大学
- 分词结果：['æĪij', 'çĪ±', 'ä,ÑåL½', 'ç § ijåÑ|æĪGæI', 'åø § åÑ|']

Qwen3的BPE分词实际上**字节级BPE**，这些拆分的tokens实际上是字节级的，所以只是看起来像乱码

重新解码：['我', '爱', '中 国', '科学 技术', '大学']

张昱：《编译原理和技术(H)》语言的描述-词法



分词算法：WordPiece

□ 起源：Google BERT

□ 原理

基于**概率最大化**原则。给定一个词，选择的子词划分应使得训练语料中整体概率最大。

公式： $\text{score} = (\text{freq_of_pair}) / (\text{freq_of_first_element} * \text{freq_of_second_element})$

□ 优点

- 对低频词能合理拆分
- 分词结果稳定，不容易产生奇怪的片段



分词策略：SentencePiece

- 起源：由 Google 提出
- 原理：自上而下
 - 首先用 **巨大的种子词汇表** 初始化
 - **逐步移除** 对整体分词质量贡献最小的词汇，直到词汇表缩小到目标大小
- 可以基于 BPE 或 Unigram 语言模型，直接在原始文本（无需预分词）上训练
- 优点
 - 与语言无关，不依赖空格作为分词边界
 - 常用于多语言模型，支持中文、日文等没有空格的语言



Tokenizer的工作流程

■ 规范化Normalization

清理文本，如统一转换为小写、去除多余空格、Unicode规范化等

■ 预分词Pre-tokenization

“Don't worry.” -> [“Don”, “'”, “t”, “ worry”, “.”]

根据简单规则(如空格和标点符号)将文本初步分割成“单词”或“粗粒度”的片段

■ 分词Tokenization

[“ worry”] -> [“ wor” , “ ry”] (BPE示例)

应用上述BPE/WordPiece算法，将预分词后的片段进一步分成最终的子词Tokens

■ 编码Encoding

[“Don” , “ ‘ “ , “t” , “ wor” , “ ry” , “.”] ->
[1234, 123, 456, 789, 2345, 99]

将每个Token映射到词汇表中对应的唯一整数ID

■ 添加特殊Tokens

根据需要添加模型使用的特殊Tokens

解码：是逆过程，将一串Token IDs转换回字符串。

将 “ wor” 和 “ ry” 正确地拼接成 “worry” ，而不是 “ worry”

张昱：《编译原理和技术(H)》语言的描述-词法



分词挑战

□ 中文分词

词间没有空格，预分词步骤需要额外的中文分词工具(如[jieba](#))，
或者直接依赖强大的子词算法(如SentencePiece)将每个汉字视为一个初始符号

□ 一些挑战

- 不一致性：同一个词在不同上下文中可能有不同的分词方式
- 长度不对等：Token序列的长度与原始字符串的长度（如字符数）没有固定比例。一个中文字符可能被分成1个或多个Tokens（罕见词）
- 多语言支持：设计一个能平衡多种语言效率的词汇表非常困难。

通常，像GPT-4这样的模型，其词汇表中会包含大量中、英、法、德等各种语言的常见子词



研究示例：张量程序自动调优的代价模型

Schedule Primitives 3

```
split i=256 to i.0=2, i.1=4, i.2=2, i.3=16
split j=512 to j.0=32, j.1=4, j.2=2, j.3=2
split k=128 to k.0=16, k.1=8
...
reorder i.0, j.0, i.1, j.1, k.0, i.2, ...
compute k.0 at j.1
fuse i.0, j.0, i.1, j.1 to i.0@j.0@i.1@j.1@
parallel i.0@j.0@i.1@j.1@
fuse ax0, ax1 to ax0@ax1@
parallel ax0@ax1@
vectorize j.3
```

预测

代价
0.83

Tensor Program 3

```
parallel i.0@j.0@i.1@j.1@ (0,1024)
  for k.0 (0,16)
    for i.2 (0,2)
      for j.2 (0,2)
        for k.1 (0,8)
          for i.3 (0,16)
            vectorize j.3 (0,2)
            T_dense = ...
parallel ax0@ax1@ (0,131072)
  T_relu = ...
```

测量

split i=256 to i.0=2, i.1=4, i.2=2, i.3=16 保留串和数字
split, i, 256, i.0, 2, i.1, 4, i.2, 2, i.3, 16
1, 0, 0, 0, ..., 0, 0, 1, 256, 2, 2, 3, 4, 4, 2, 5, 16 编码

(PrimitiveSequence) $S ::= p^*$
(Primitive) $p ::= \tau (id \mid num)^*$
(PrimitiveType) $T \ni \tau ::= split \mid reorder \mid fuse \mid \dots$
(NameParam) id
(Number) num

(a) Abstract Schedule Primitive Sequence

(Features) $f = F(p) ::= F_1(\tau) (F_2(id) \mid F_3(num))^*$
 $F : \text{Primitive} \rightarrow \text{Features}$
 $F_1 : \text{PrimitiveType} \rightarrow \text{OnehotVector}$
 $F_2 : \text{NameParam} \rightarrow \text{Token}$
 $F_3 : \text{Number} \rightarrow \text{Number}$

(b) TLP Extractor

Type One-hot Table

split	1	0	0	0	...	0	0
reorder	0	1	0	0	...	0	0
compute	0	0	1	0	...	0	0
fuse	0	0	0	1	...	0	0
...	...	...	...	...	...	...	...
parallel	0	0	0	0	...	1	0
vectorize	0	0	0	0	...	0	1

Character Tokens

i	1	j.3	10
i.0	2	k	11
i.1	3	k.0	12
i.2	4	k.1	13
i.3	5	i.0@j.0@i.1@j.1@	14
j	6	ax0	15
j.0	7	ax1	16
j.1	8	ax0@ax1@	17
j.2	9	...	...

TLP-特征提取
调度原语序列 → 特征



多模态大模型的分词

把不同模态的原始输入映射为统一的**离散表示**(token 序列)

图像/语音/视频等是连续模态，需将像素/波形等连续信号**编码**成离散 token

■ 图像 Tokenizer

- 使用向量量化，将图像分块编码到离散的codebook(码本，即词典)，如Imagen、Stable Diffusion
- 图像切成patch，再编码成token，如ViT Patch Embedding+离散化、Clip(直接embedding)
- 分层量化：多级量化器编码图像，如RQ-VAE等

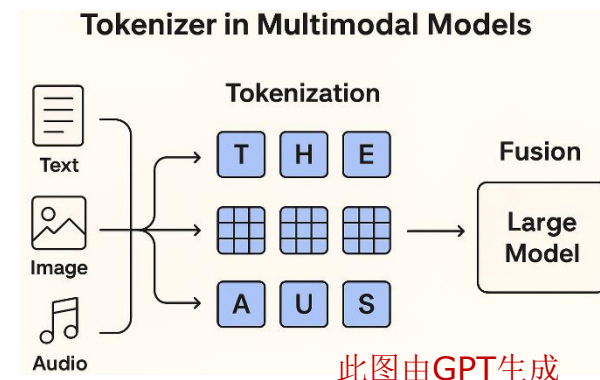
■ 音频 Tokenizer

- 把原始波形压缩成离散码字序列，常用于音乐生成（MusicLM、AudioLM）
- Residual Vector Quantization，高效表示音频信号

■ 视频（时序图像） Tokenizer

- 图像 Tokenizer + 时间建模（逐帧离散化，再建模时序关系）
- 3D VQ-VAE：直接在时空域做离散化

■ 多模态统一Tokenizer：让不同模态共享词汇表





多模态大模型的分词

□ 一些挑战

- **模态的差异性：**如何权衡**压缩信息量**和**保持语义细节**
- **Token粒度选择：**如何权衡粒度与效率
 - 图像：patch太小→token数过多； patch太大→丢失局部信息
 - 音频：时间片过短→序列过长；时间片过长→难捕捉细节（音色、情感等）
序列长度与计算开销
 - 图像/视频 token 数量远超文本；视频包含时间维度，token 序列可能是文本的 数百倍
- **跨模态语义对齐：**如何在统一空间中互相理解
 - 不同模态的 token 在信息含义和统计分布上差异巨大
 - 如何让“文本词元”和“图像 patch token”在共享空间里表示相近语义
- **信息压缩与保真：避免 token 化带来的语义缺失**
 - 离散化不可避免带来信息丢失：丢失纹理、音色等；对下游生成与理解任务都会造成影响



开放作业

1. 现实世界中的领域语言及解析

- 科学：数学、化学、物理、生物、地理、.....
- 产业：工业制造、机器人、视觉、.....

2. 大模型中的词元是什么，怎么从输入解析的

- 不同模态的输入
- 不同的大模型
- 分词算法